

プログラミング演習スライド

金子知適

東京大学総合文化研究科 (kaneko@acm.org)

week3

print文 と混ぜるな危険

プログラミング演習
スライド

金子

for

ループ不変条件

while

評価 v.s. print

セル実行: 最後の式の評価[†] とそれまで print した内容が **混ざって** 表示される
[†]ただし None 以外

関数の return v.s. print

— return: 式の構成要素 —

```
def add(a, b):  
    return a+b    # 基本パターン
```

- `add(3, 5) → 8`
- `add(3, 5) + 7 → 15`
- `add(1000, add(3, 5) + 7) → 1015`

— print: 外部に出力 —

```
def add(a, b):  
    print(a+b)    # 間違い
```

- `add(3, 5) → 8 (と表示)`
- `add(3, 5) + 7 → 8 (と表示)後にエラー`
- `add(1000, add(3, 5) + 7) → 8 (と表示)後にエラー`

print の結果は、外部 (ディスプレイ, プリンタ, テスター) に送られる。
プログラムから再度利用することはできない

[重要!] for を使う関数 — 範囲の全てに何かする —

プログラミング演習
スライド

金子

for
関数と for

ループ不変条件

while

関数の基本パターン (for)

```
def 関数名(引数1,...):  
    変数 = 式      # 初期値  
    for i in range(...):  
        変数 = 式      # 値を更新  
    return 変数
```

例 0 から $n-1$ までの和

```
def sum_to_n(n):  
    total = 0  
    for i in range(n):  
        total = total + i  
    return total
```

考え方 — $n = 4$ の具体例, i.e., $0+1+2+3$ —

部分和 s_i (0から i までの和) で表現

```
def sum3():  
    se = 0 # 空集合の和  
    s0 = se + 0 # 0 (の和)  
    s1 = s0 + 1 # [0,1] の和  
    s2 = s1 + 2 # [0,1,2] の和  
    s3 = s2 + 3 # [0,1,2,3] の和  
    return s3
```

部分和 s_i をまとめて total (略記 t) で表現

```
def sum3():  
    t = 0      # t は total の略記, 初期値 se  
    t = t + 0 # t は s0 相当に  
    t = t + 1 # t は s1 相当に  
    t = t + 2  
    t = t + 3  
    return t   # t は s3 相当
```

Q. ところで, なぜ 0 を足すの?

A. 0 を無視できないケース, たとえば, $f(0)$ から $f(n-1)$ の和などに, このパターンを応用するため

[重要] ループ不変条件

(授業範囲内, 試験範囲外)

プログラミング演習
スライド

金子

for

ループ不変条件

while

関数の基本パターン (for)

```
def 関数名(引数1,...):  
    変数 = 式      # 初期値  
    for i in range(...):  
        # (a) 更新 前 の i と変数の関係  
        変数 = 式      # 値を更新  
        # (b) 更新 後 の i と変数の関係  
    return 変数
```

例 0 から n-1 までの和

```
def sum_to_n(n):  
    total = 0  
    for i in range(n):  
        # (a) total = 0..i-1 の和  
        total = total + i  
        # (b) total = 0..i の和  
    return total
```

Best practice: **ループ不変条件** (a), (b) どちらかをコメントに書こう! (平常点)

考え方: $3 = (4-1)$ までの合計, i.e., $0+1+2+3$

補助変数 s_i (0からiまでの和) の表現

```
def sum3():  
    se = 0 # 空集合の和  
    s0 = se + 0 # 0 (の和)  
    s1 = s0 + 1 # [0, 1] の和  
    s2 = s1 + 2 # [0, 1, 2] の和  
    s3 = s2 + 3 # [0, 1, 2, 3] の和  
    return s3
```

補助変数 s_i を一つの変数 total で表現

```
def sum3():  
    total = 0 # totalは se 相当  
    total = total + 0 # total は s0 相当に  
    total = total + 1 # total は s1 相当に  
    total = total + 2  
    total = total + 3  
    return total # total は s3 相当
```

演習 様々なパターン

プログラミング演習
スライド

金子

for

ループ不変条件

while

例 0 から $n-1$ までの2乗和

```
def squared_sum_to_n(n):  
    total = 0  
    for i in range(n):  
        # (a) 0 (i == 0)  
        # (a) 0..i-1 の2乗和 (i > 0)  
        total = total + i*i  
        # (b) 0..i の2乗和  
    return total
```

例 文字列の配列 seq の連結

```
def concat_of(seq):  
    ans = ''  
    for i in range(len(seq)):  
        # (a) '' (i == 0)  
        # (a) seq[i-1] までの連結 (i>0)  
        ans = ans + seq[i]  
        # (b) seq[0]..seq[i] の連結  
    return ans
```

例 整数の配列 seq の要素の和

```
def sum_of(seq):  
    total = 0  
    for i in range(len(seq)):  
        # (a) 0 (i == 0)  
        # (a) seq[i-1] までの和 (i>0)  
        total = total + seq[i]  
        # (b) seq[0]..seq[i] の和  
    return total
```

例 配列の作成

```
def iota(n):  
    ans = []  
    for i in range(n):  
        # (a) [] (i == 0)  
        # (a) [0,..,i-1] (i>0)  
        ans = ans + [i]  
        # (b) [0,..,i]  
    return ans
```

演習 様々なパターン (2) 論理演算

プログラミング演習
スライド

金子

for

ループ不変条件

while

$\forall, \exists, \neg\forall, \neg\exists$ を作ってみよう. 引数 `seq` は, `bool` の列とする.

`all_true(seq)` と `not any_false(seq)`, `all_false(seq)` と `not any_true(seq)` は等価.

全要素が True

```
def all_true(seq):
    all_ok = True
    for i in range(len(seq)):
        all_ok = all_ok and seq[i]
        # (b) 先頭から i 番目まですべて真
    return all_ok
```

全要素が False

```
def all_false(seq):
    any_ok = False
    for i in range(len(seq)):
        any_ok = any_ok or seq[i]
        # (b) 先頭から i 番目まで1つでも真
    return not any_ok
```

1つでも True

```
def any_true(seq):
    any_ok = False
    for i in range(len(seq)):
        any_ok = any_ok or seq[i]
    return any_ok
```

1つでも False

```
def any_false(seq):
    all_ok = True
    for i in range(len(seq)):
        all_ok = all_ok and seq[i]
    return not all_ok
```

初期値いろいろ 0, '', [], True, False

while 文のくり返し実行 — 回数指定無し —

while の基本パターン

```
while 論理式:  
    ブロック
```

- 意味 ブロックを、「論理式」が真のあいだ実行
- **厳密さ注意** ブロックのインデントは厳密に 2 インデント ~ 行頭の空白文字の数
- **厳密さ注意** 行末のコロン : を忘れずに (半角, ASCII文字)

例: for と同じ動作

```
for i in range(3):  
    print('Hi, there!')
```

```
cnt = 0  
while cnt < 3:  
    print('Hi, there!')  
    cnt = cnt + 1
```

```
for i in range(3):  
    print(i)
```

```
cnt = 0  
while cnt < 3:  
    print(cnt)  
    cnt = cnt + 1
```


